From Sparse Ratings to Movie Discovery: Matrix Factorization on MovieLens 100K

Student: Polina Maltseva | Team: Individual | Email: pdmaltseva@edu.hse.ru | Date: 2026-10-16
Assignment: A05 - From Ratings to Recommendations (Week 5)

Abstract

A streaming service must turn a 93.70% sparse rating matrix into short, useful movie lists. I implemented a 16-factor TensorFlow.js model with chronological per-user splitting, Adam training, validation checkpointing, Top-10 retrieval, and Recall@10 evaluation. The selected checkpoint reached test RMSE 1.0503 versus 1.2289 for the training-mean baseline, while Recall@10 was 0.0134 over 861 evaluable users. The experiment shows that accurate rating prediction can still produce weak and sometimes poorly personalized discovery lists, especially for light users.

Index Terms - matrix factorization, collaborative filtering, TensorFlow.js, MovieLens, Recall@10, novelty.

1. Introduction

MovieLens 100K contains explicit ratings but leaves most user-movie pairs unobserved. Treating those missing pairs as zero would convert absence of evidence into negative preference; instead, the model optimizes only observed training ratings. The operational problem is not only to predict held-out ratings, but also to form a useful Top-10 row from the catalogue.

A concrete example is User 1: from 217 training and 27 validation ratings, the application returns a list led by *Thieves (Voleurs, Les)* with score 4.8804. Two items in that list are relevant held-out movies, so this user's Recall@10 is 2/19 = 0.1053.

Contributions. (1) A complete implementation of the six required functions using batched TensorFlow.js operations. (2) A reproducible experiment with two identical seeded runs, one regularization change, two user-level analyses, and a stored machine-readable evidence file.

2. Related Work

The dataset is the stable MovieLens 100K benchmark [1], [2]. Matrix factorization represents users and items in a shared latent space and scores a pair with a dot product [3]. Adam supplies adaptive first-order updates for the sparse mini-batches [4]. TensorFlow.js provides browser-side tensor operations and automatic differentiation [5]. Following Herlocker et al. [6], I report both rating error and a list metric because predictive accuracy alone does not characterize recommendation value.

Alternatives considered

Item-item similarity: interpretable, but sparse overlap can make similarities unstable. **ALS:** natural for factor models, but the assignment requires TensorFlow.js optimization. **User and item biases:** likely to stabilize ranking, but excluded from the required baseline formula so that the six TODO functions remain comparable to the starter contract.

3. Data and Recommendation Analysis

Quantity	Observed value	Interpretation
Users / movies / ratings	943 / 1,682 / 100,000	Raw MovieLens 100K files
Matrix sparsity	93.70%	1 - 100,000 / (943 x 1,682)
Train / validation / test	79,619 / 9,596 / 10,785	Per user, ordered by timestamp
Training mean	3.5806	Mean-rating baseline prediction
Absent from training	0 users / 71 movies	Cold movie ids cannot be learned
Unseen val / test rows	35 / 53	Ratings whose movie has no train row
Recall evaluation	861 users / 5,142 relevant	82 users excluded; 13 relevant unseen ids

Table I. Dataset, chronological split, and evaluation exclusions.

3.1 Two Top-10 lists

#	User 1	Score	n	#	User 19	Score	n
1	Thieves (Voleurs, Les) (1996)	4.880	6	1	Hellraiser: Bloodline (1996)	5.079	9
2	Saint of Fort Washington, The (1993)	4.809	1	2	Mr. Magoo (1997)	5.034	8
3	A Chef in Love (1996)	4.802	8	3	Super Mario Bros. (1993)	5.015	15
4	Persuasion (1995)	4.799	38	4	Speed 2: Cruise Control (1997)	4.983	26
5	When the Cats Away (Chacun cherche son chat) (1996) *	4.791	11	5	Wishmaster (1997)	4.943	20
6	Wrong Trousers, The (1993) *	4.759	101	6	Thinner (1996)	4.904	33
7	When We Were Kings (1996)	4.752	34	7	Flipper (1996)	4.896	14
8	Dream With the Fishes (1997)	4.727	4	8	Dracula: Dead and Loving It (1995)	4.863	10
9	Perfect Candidate, A (1996)	4.685	3	9	Fluke (1995)	4.845	9
10	Close Shave, A (1995)	4.672	96	10	Specialist, The (1994)	4.834	11

*Table II. Top-10 recommendations from the default checkpoint. n = training ratings; * = relevant test movie. No item appears in the corresponding user's train or validation history.*

3.2 Familiar choices vs. new discoveries

User 1 (heavy user). The history is led by Drama (38.1%), Comedy (33.2%), and Action (28.3%). The list keeps Comedy (40%) and Drama (30%), and the two hits - *When the Cats Away* and *The Wrong Trousers* - fit that familiar profile. Yet the median popularity drops from 106 training ratings in the history to 9.5 in the list. This is discovery inside familiar genres: safer than a completely novel direction, but strongly exposed to noisy estimates for rare films.

User 19 (light user). Only 18 history ratings are available; Comedy and Drama each occur in 55.6% of them. The list shifts to Action and Horror (40% each), including *Hellraiser: Bloodline*, *Wishmaster*, and *Thinner*. Its median item has 12.5 training ratings versus 193.5 in the history, and the only eligible relevant test movie is missed. This is novel but not convincingly personal. For a streaming platform, such a row should be constrained by confidence or mixed with a popularity/bias component.

4. Method

4.1 Pipeline

1) Parse `u.data` and `u.item`. 2) Group ratings by user, sort by timestamp (movie id breaks ties), and allocate floor(0.8n), floor(0.1n), and the remainder to train, validation, and test. 3) Initialize 16-dimensional user and movie factors. 4) Train with mini-batched Adam on observed ratings only. 5) retain the earliest epoch with minimum validation RMSE. 6) Produce Top-10 lists excluding the user's train and validation items. 7) evaluate Recall@10 for test ratings at least 4.0 whose movie id occurs in training.

4.2 Model and loss

The prediction is **r_hat(u,i) = mu + p_u dot q_i**, where mu is the training mean and both vectors have 16 components. Displayed rating predictions are clamped to [1,5], but training uses the unclamped score so gradients do not vanish at the boundaries. For each mini-batch, the objective is mean squared error plus 0.05 times the mean squared norm of the gathered user and movie rows. Applying a full-matrix penalty at every batch would repeatedly regularize unrelated rows and overwhelm the data term.

4.3 Training and checkpointing

Adam uses learning rate 0.01, batch size 64, 20 epochs, and deterministic seed 20261016. User and item matrices use adjacent seeds to avoid identical same-id rows. A seeded Fisher-Yates shuffle changes the mini-batch order each epoch. Whenever validation RMSE improves, both factor matrices are cloned; the selected matrices are restored after epoch 20 so test metrics and recommendations use the chosen checkpoint rather than the final weights.

4.4 Retrieval and metric

For a user, one matrix multiplication scores all movie rows, a mask assigns -1e9 to missing metadata and train/validation items, and `tf.topk` returns the highest remaining scores. Recall for a user is hits divided by eligible relevant test movies; the reported metric is the unweighted mean across 861 evaluable users. Thirteen distinct relevant test movie ids absent from the training pool are reported as exclusions rather than silently counted as recommendable.

4.5 Implementation and environment

Component	Version / setting
TensorFlow.js	4.22.0; webgl backend
Browser	Headless Chrome 155.0.0.0
Renderer	ANGLE Metal Renderer: Apple M4 Pro
Factors / epochs / batch	16 / 20 / 64
Learning rate / L2 / seed	0.01 / 0.05 / 20261016

Code availability: https://github.com/PoliMa979/week5. The repository includes the application, browser runner, raw evidence JSON, screenshot, report source, and this PDF.

5. Experiments and Verification

Model	Selected epoch	Val RMSE	Test RMSE	Recall@10
Training-mean baseline	-	1.1830	1.2289	-
MF, L2 = 0.05	2	1.0042	1.0503	0.0134
MF, L2 = 0.10	6	0.9987	1.0444	0.0080

Table III. Rating and retrieval results from validation-selected checkpoints.

The default model improves test RMSE by 0.1786 (14.5%) relative to the mean baseline. Its Recall@10 is only 0.0134: 102 hits over 5,142 eligible relevant movies, with at least one hit for 87 of 861 users. L2 = 0.10 improves test RMSE to 1.0444 but reduces Recall@10 to 0.0080. Therefore the one-variable change helps rating prediction and hurts list retrieval; optimizing RMSE alone is not sufficient.

Epoch	Train RMSE	Validation RMSE
1	0.8830	1.0256
2	0.7995	1.0042
3	0.7543	1.0049
5	0.7231	1.0184
10	0.7037	1.0263
15	0.6998	1.0308
20	0.6964	1.0317

Table IV. Selected points from the default learning curve. Validation error is best at epoch 2 while training error continues falling, which motivates checkpoint restoration.

5.1 Verification

Automated checks. The starter harness reports 11 passed, 0 failed, and 0 pending. It tests allocation, clamping, batched training, earliest-tie checkpoint selection, history masking, Recall@10, and zero-denominator guards. Two complete default runs produced identical 20-epoch histories; the maximum absolute difference was 0. Tensor count was stable at 27 after each of the three full runs.

5.2 Independent dot-product check

k	p_k	q_k	product
1	-0.1349	0.2726	-0.0368
2	0.0606	0.1159	0.0070
3	-0.4743	-0.6449	0.3059
4	0.1724	0.2583	0.0445
5	0.2249	0.2940	0.0661
6	0.3534	0.6537	0.2310
7	0.5944	0.4166	0.2476
8	0.2221	-0.2033	-0.0452

k	p_k	q_k	product
9	-0.3668	-0.4511	0.1655
10	-0.5033	-0.1788	0.0900
11	-0.1845	-0.3807	0.0702
12	-0.1867	-0.6037	0.1127
13	0.0703	-0.2958	-0.0208
14	0.2598	-0.0624	-0.0162
15	0.0374	-0.3426	-0.0128
16	-0.2315	-0.3927	0.0909

For User 1 and *Thieves (Voleurs, Les) (1996)*, the 16 products sum to 1.2998. Adding mu = 3.5806 gives 4.8804; the page score is 4.8804. The difference is below 4e-7 and is explained by Float32 rounding.

6. Discussion

6.1 Failure case, root cause, and fix

The first evidence run appeared to leak two tensors per completed experiment: the count increased from 29 to 31 to 33. The model code was initially suspect, but the leak came from the verification runner: two factor slices were read for the manual dot-product check outside `tf.tidy`. Wrapping those two reads in `tf.tidy` and repeating all three experiments held the count at 27 after every run. This distinguished an instrumentation defect from a training-loop defect and verified that repeated Train clicks dispose the old factor variables.

6.2 What worked

- The validation-selected model consistently beats the mean-rating baseline.
- Seeded initialization and shuffling make complete histories exactly reproducible.
- Restoring epoch 2 prevents the final epoch's worse validation state (1.0317) from driving the Top-10 lists.
- User histories are masked correctly: both inspected lists have zero train/validation overlaps.

6.3 What was surprising

- Better RMSE did not imply better retrieval: L2 = 0.10 reduced test RMSE by 0.0059 but lost 43 of the default run's 102 hits.
- Discovery is often driven by low-support movies. The default model puts *Pather Panchali* (7 training ratings) in 204 of 943 lists and *Perfect Candidate* (3 ratings) in 177 lists.
- User 19's list is highly novel but poorly aligned with the Comedy/Drama-heavy history, illustrating how a short user vector can amplify uncertain item directions.

6.4 Next improvement

I would add explicit user and movie biases, keep L2 = 0.05 as the retrieval-oriented baseline, and rerank with a confidence rule such as a minimum of 20 training ratings for discovery slots. The comparison should report RMSE, Recall@10, catalogue coverage, and the share of Top-10 slots occupied by low-support movies. This directly tests whether rare-item exposure becomes safer without collapsing discovery into popularity.

7. AI Usage Disclosure

Tool. OpenAI Codex desktop 26.924.22138 (build 11645), used on 2026-10-16. **Uses.** Requirement extraction, repository comparison, implementation of the six TODO functions, browser automation, experiment logging, reference checking, and report drafting. **Verification performed in the assisted session.** 11/11 starter tests, two identical default training histories, a one-variable regularization run, stable tensor counts, train/validation masking, and a direct 16-term dot-product recomputation from the learned factors. **Not independently verified by the student at report-generation time.** The student has not yet repeated the manual addition or rerun the browser experiment without Codex assistance; this limitation is disclosed rather than replaced with a false claim. **Session log.** `polina_maltseva_a05_session.json` must be the native, unmodified Codex export. No synthetic session file was created.

Evidence files: `evidence/experiment_results.json` contains every metric, history row, recommendation, exclusion, and factor vector used here; `evidence/trained_application.png` records the running page.

References

[1] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, Art. 19, 2015. doi: 10.1145/2827872.

[2] GroupLens Research, "MovieLens 100K Dataset." [Online]. Available: https://grouplens.org/datasets/movielens/100k/. [Accessed: 2026-10-16].

[3] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30-37, Aug. 2009. doi: 10.1109/MC.2009.263.

[4] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. doi: 10.48550/arXiv.1412.6980.

[5] D. Smilkov *et al.*, "TensorFlow.js: Machine Learning for the Web and Beyond," in *Proc. 2nd SysML Conf.*, 2019. [Online]. Available: https://arxiv.org/abs/1901.05350. [Accessed: 2026-10-16].

[6] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. Riedl, "Evaluating Collaborative Filtering Recommender Systems," *ACM Transactions on Information Systems*, vol. 22, no. 1, pp. 5-53, Jan. 2004. doi: 10.1145/963770.963772.

[7] TensorFlow.js, "API Reference, version 4.22.0." [Online]. Available: https://js.tensorflow.org/api/4.22.0/. [Accessed: 2026-10-16].

Reproducibility note

All numeric claims in Tables I-IV and the two Top-10 lists are generated from `evidence/experiment_results.json`. The browser runner performs the tests, three training runs, the user-level analysis, catalogue-frequency analysis, and dot-product extraction in one command. The report builder reads that JSON rather than duplicating numbers by hand.